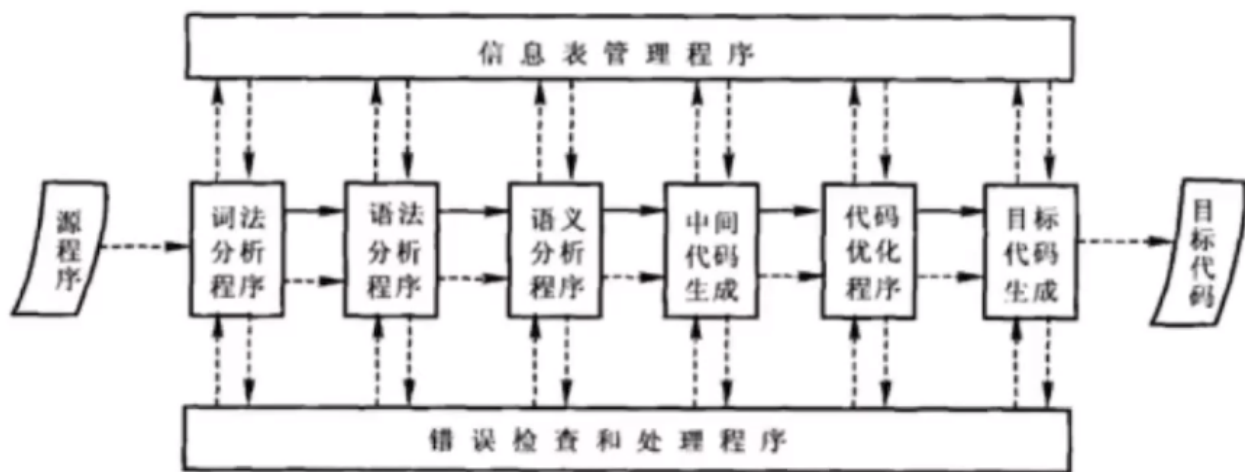


绪论

编译系统由哪些部分组成



词法分析程序、语法分析程序、语义分析程序、中间代码生成

代码优化程序、目标代码生成。

前后文无关文法和语言

基础定义

2-1 设有字母表 $A_1 = \{a, b, \dots, z\}$, $A_2 = \{0, 1, \dots, 9\}$, 试回答下列问题:

- (1) 字母表 A_1 上长度为 2 的符号串有多少个?
- (2) 集合 $A_1 A_2$ 含有多少个元素?
- (3) 列出集合 $A_1 / (A_1 \cup A_2)^*$ 中的全部长度不大于 3 的符号串。

$\Delta A+B = \{w | w \in A \text{ 或 } w \in B\}$ 元素和

$AB = \{xy | x \in A \text{ 且 } y \in B\}$ 有顺序

$\Delta A^0 = \{\epsilon\}$ $A^1 = A$ $A^2 = AA$ $A^3 = \dots$

正闭包 $A^+ = \bigcup_{i=1}^{\infty} A^i = A^1 + A^2 + A^3 + \dots + A^i$

自反传递闭包 $A^* = \bigcup_{i=0}^{\infty} A^i = A^0 + A^1 + A^2 + \dots + A^i$ $A^+ + \{\epsilon\} = A^*$

$$(1) 26 \times 26 = 676 \text{ 个}$$

$$(2) 26 \times 10 = 260 \text{ 个}$$

$$(3) 26 + 26 \times 26 + 26 \times 26 \times 26 = 34658 \text{ 个}$$

$A+B$ 为元素和, AB 是对应元素有顺序相乘, 正闭包比自反传递闭包少个 A^0 空集。

文法表示

2-2 试分别构造产生下列语言的文法。

(1) $\{a^n b^n | n \geq 0\}$; $\{S \rightarrow aSb | \epsilon\} = \{S \rightarrow aSb, S \rightarrow \epsilon\}$

(3) $\{a^n \# b^n | n \geq 0\} \cup \{c^n \# d^n | n \geq 0\}$; $S \rightarrow X | Y$ $X \rightarrow aXb | \#$ $Y \rightarrow cYd | \#$

(5) 任何不是以 0 开始的所有奇整数所组成的集合; $S \rightarrow +2|-2$ $z \rightarrow A | B \notin A$

$A \rightarrow 1|3|5|7|9$ $B \rightarrow A|2|4|6|8$ $C \rightarrow \frac{B}{0} | \epsilon | CC$

$\Delta \text{文法 } G[S] = (V_N, V_T, P, S)$

- V_N : 非终结符号集 $A, B, \dots$
- V_T : 终结符号集 $a, b, c, \dots$
- P : 产生式集 $S \rightarrow A$
- S : 开始符号

(1) $G[S] = (\{S\}, \{a, b\}, \{S \rightarrow aSb | \epsilon\}, S)$

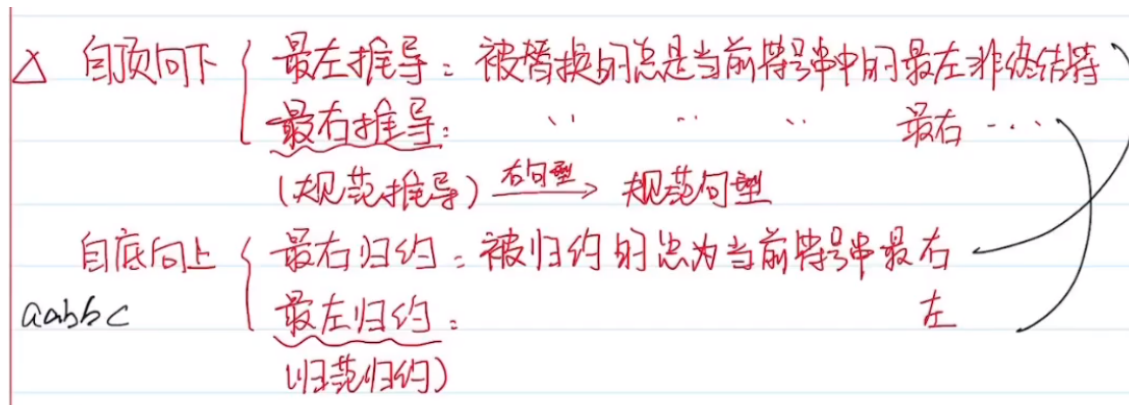
(3) $G[S] = (\{S, X, Y\}, \{a, b, c, d, \#\}, \{S \rightarrow X | Y, X \rightarrow aXb | \#, Y \rightarrow cYd | \#\}, S)$

(5) $G[S] = (\{S, z, A, B, C\}, \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -\}, \{S \rightarrow +2|-2, z \rightarrow A | B \notin A$

$A \rightarrow 1|3|5|7|9, B \rightarrow A|2|4|6|8, C \rightarrow \frac{B}{0} | \epsilon | CC\}, S)$

非终结符号可以自己定义, 尤其是在分块较为复杂的时候常会用到。终结符号是确定了的, 很好写。产生式集, 首先是确定哪里需要套娃, 然后将非终结符号插入套娃的地方表示, 要在后面或上个终结符号。

推导、归约与语法树



推导和归约为逆过程，最右推导为规范推导，最左归约为规范归约。

△ 语法树

子树：任一结点及其全部后继

直接子树 (树高为1)：-子树根只有直接后继

一棵语法树 $\leftrightarrow$ 多种推导序列

一棵语法树 $\leftrightarrow$ 唯一最左推导和最右推导

在做题的时候先写语法分析数，再做左右推导可以更便于写出。写推导式按照统一非结束文法符号比较，然后先观察句子的首尾。

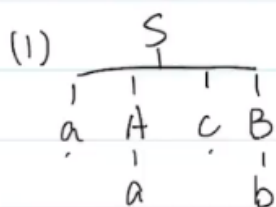
2-7 设已给文法 $G[S]$ ：

$S \rightarrow aAcB$ $S \rightarrow BdS$ $B \rightarrow aScA$ $B \rightarrow cAB$
 $A \rightarrow BaB$ $A \rightarrow aBc$ $A \rightarrow a$ $B \rightarrow b$

试检验下列符号串中哪些是 $G[S]$ 中的句子，给出这些句子的最左推导、最右推导和相应的语法树。

(1) aacb

(2) aabacbadcd



最左推导： $S \Rightarrow aAcB \Rightarrow aacB \Rightarrow aacb$

最右推导： $S \Rightarrow aAcB \Rightarrow aAcb \Rightarrow aacb$

(2) 不是 $G[S]$ 的句子

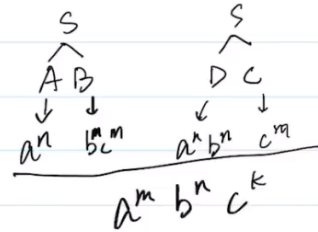
二义性文法、短语、句柄

定义：某个句子对应不只一个最左/最右推导。相反，无二义性文法为该文法所产生的每一个句子仅有一颗语法数。

2-10 试证明：文法

$$\begin{array}{ll} S \rightarrow AB & S \rightarrow DC \\ B \rightarrow bBc & B \rightarrow bc \\ D \rightarrow aDb & D \rightarrow ab \\ A \rightarrow aA & A \rightarrow a \\ C \rightarrow cC & C \rightarrow c \end{array}$$

为二义性文法。



△ 二义性文法：L(G) 的某个句子对应不只一个最左/最右推导

△ 无二义性文法：该文法所产生的每个句子都仅有一颗语法树

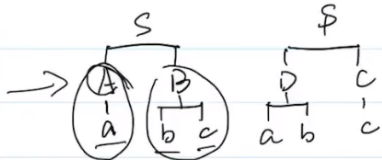
证：A 可推出 $a, aa, \dots$

B $\dots bc, bbcc, bbbccc \dots$

C $\dots c, cc, ccc \dots$

D $\dots ab, aabb, aaabbb \dots$

有 abc 对应 2 棵不同的语法树



$S \Rightarrow AB \Rightarrow Abc \Rightarrow abc$

$S \Rightarrow DC \Rightarrow Dc \Rightarrow abc$

$\therefore$ 该文法具有二义性

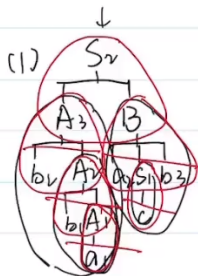
证明是否是二义性文法，找出反例证伪即可。

△ 短语：每棵子树的叶子

直接短语：每棵直接子树的叶子

句柄：某句型的最左直接短语（即规范分析中最先被归约的子串）

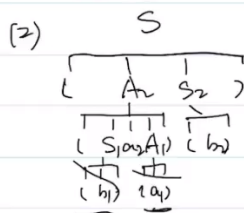
素短语：至少包含一个终结符且不包含更小素短语的短语



最右推导： $S \Rightarrow AB \Rightarrow AaB \Rightarrow Aacb \Rightarrow bAacb \Rightarrow bbAacb \Rightarrow bbaacb$

短语： $a, b_1a_1, b_2b_1a_1, c, a_2cb_3, b_2b_1a_1a_2cb_3$

句柄： $a_1, b_1A_1, b_2A_2, c_1, a_2S_1b_3, A_2B$



最右推导： $S \Rightarrow (AS_1) \Rightarrow (A(b_1)) \Rightarrow ((S_1A_1)(b_1)) \Rightarrow (((S_1a_1)(b_1))(b_2))$

短语： $(b_1), (a_1), (b_1)a_2(a_1), (b_2), ((b_1)a_2(a_1)(b_2))$

句柄： $(b_1), (a_1), S_1a_2A_1, (b_2), (A_2S_2)$

先画出语法树，从S开始画，然后是链接往下写非终止符号，指导全部叶子为终止符。短语是从倒数第二层开始，依次找他们的叶子结点，逐步向上往上拼接。句柄和短语的区别：句柄内有非终止符了，不用像短语那样拼接。

文法化简

2-13 化简下列各个文法。

(1) $S \rightarrow aABS$ $S \rightarrow bCACd$ $A \rightarrow bAB$ $A \rightarrow cSA$
 $A \rightarrow cCC$ $B \rightarrow bAB$ $B \rightarrow cSB$ $C \rightarrow cS$
 $C \rightarrow c$

(3) $S \rightarrow ac$ $S \rightarrow bA$ $A \rightarrow cBC$ $B \rightarrow SA$ $A \rightarrow cSAC$
 $C \rightarrow bC$ $C \rightarrow d$

△化简文法

① 从S开始一步步看，将含有永远消不掉的符号的产生式消去
 (无用符号、无用产生式) ✓

② 删 ϵ -产生式 $A \rightarrow \epsilon$

③ 删 形如 $A \rightarrow B$ 的单一产生式

词法分析及词法分析程序

状态转换图

3-6 试构造一右线性文法,使得它与如下的文法等价

$S \rightarrow AB$ $A \rightarrow UT$ $U \rightarrow a|aU$

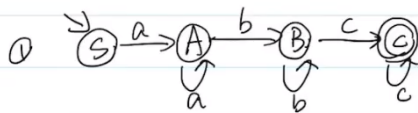
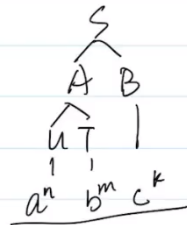
$T \rightarrow b|bT$ $B \rightarrow c|cB$

并根据所得的右线性文法,构造出相应的状态转换图。

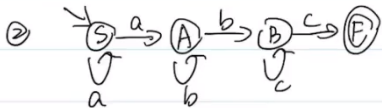
△右线性文法: 仅有形如 $A \rightarrow aB$ 及 $A \rightarrow a$ 的推导式

△状态转换图(右线性文法,无ε-产生式时)

$\begin{cases} A \rightarrow aB & (A) \xrightarrow{a} (B) \\ A \rightarrow a & (A) \xrightarrow{a} (\epsilon) \end{cases}$



$S \rightarrow aA$, $A \rightarrow aA|bB$, $B \rightarrow bB|cC|c$, $C \rightarrow cC|c$ (C为终态)



$S \rightarrow aS|aA$, $A \rightarrow bA|bB$, $B \rightarrow cB|c$

右线性文法是最左边是终结符。状态转换图先根据文法画出语法数, 然后根据其最下面拼接起来的句型来写状态转移图, 其中状态节点为非终结符, 终结符为过程量。

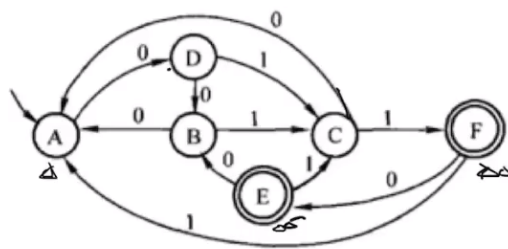
3-7 对于如题图 3-7 所示的状态转换图:

(1) 写出相应的右线性文法;

(2) 指出它接受的最短输入串;

(3) 任意列出它接受的另外四个输入串;

(4) 任意列出它拒绝接受的四个输入串。



(1) $A \rightarrow 0D$, $B \rightarrow 0A|1C$, $C \rightarrow 0A|1F|1$, $D \rightarrow 0B|1C$, $E \rightarrow 0B|1C$, $F \rightarrow 0E|1A|0$

(2) 011

(3) 0011, 0110, 000011, 0000011

(4) 任意1开头

状态转化矩阵

3-9 对于下列的状态转换矩阵：

(1) 分别画出相应的状态转换图；

(2) 写出相应的 3 型文法： 正规文法 (右线性) $A \rightarrow aB$ 及 $A \rightarrow a$

(3) 用自然语言分别描述它们所识别的输入串的特征。

	a	b
S	A	S
A	A	B
B	B	B

(i) 初态: S

终态: B



(2) $S \rightarrow aA | bS$, $A \rightarrow aA | bB | b$, $B \rightarrow aB | bB | a | b$

(3)

3型文法指的就是正规文法 (右线性)

NFA确定化和最小化

确定化

1. 求 ϵ -CLOSURE(S) 括号内初态

2. 画表格

②	I	Ia	Ib
初 q_0	$\{S\}$	$\{A\} q_1$	ϕ
q_1	$\{A\}$	ϕ	$\{S, B\} q_2$
终 (q_2)	$\{S, B\}$	$\{A\} q_1$	ϕ

把流程图想象出树的结构，根据流程图一步步往下广度遍历的方式往下走，q可以相当于标注的层数。最后用q来代替初始的ABC结点。

最小化

$$\pi = \{ \{q_2\} \{q_0, q_1\} \}$$

$$\{q_0\}a = q_1 \quad \text{可区分}$$

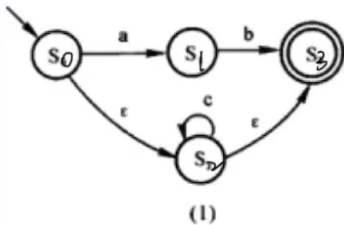
$$\{q_1\}a = \phi$$

$$\pi_{\text{new}} = \{ \{q_0\} \{q_1\} \{q_2\} \}$$



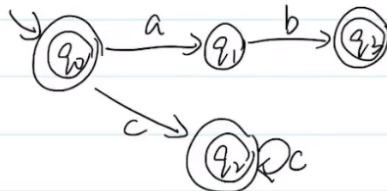
带有 ϵ 的NFA确定化

3-13 将如题图 3-13 所示的具有 ϵ 动作的 NFA 确定化。



$$\textcircled{1} \epsilon\text{-closure}(S_0) = \{S_0, S_2, S_3\}$$

$\textcircled{2} \quad I$	I_a	I_b	I_c
初 $q_0 \{S_0, S_2, S_3\}$	$\{S_1\} q_1$	ϕ	$\{S_2, S_3\} q_2$
$q_1 \{S_1\}$	ϕ	$\{S_3\} q_3$	ϕ
$q_2 \{S_2, S_3\}$	ϕ	ϕ	$\{S_2, S_3\} q_2$
$q_3 \{S_3\}$	ϕ	ϕ	ϕ



有限自动机最小化

3-14 将如题图 3-14 所示的有限自动机最小化。

	a	b
S_0 0	S_1 1	S_6 5
S_1 1	S_2 2	S_7 7
S_2 2	S_3 3	S_8 5
S_3 3	S_4 5	S_7 7
S_4 4	S_5 5	S_5 5
S_5 5	S_1 1	S_1 1
S_6 6	S_3 3	S_4 4
S_7 7	S_0 0	S_1 1
S_8 8	S_1 1	S_8 8

(1) 初态: S_0
终态: S_1, S_2, S_4, S_7

划分

$$\pi = \{ \{S_0, S_3, S_5\} \{S_1, S_2, S_7\} \}$$

$$\{S_0\}a = \{S_1\}$$

$$\{S_3\}a = \{S_5\}$$

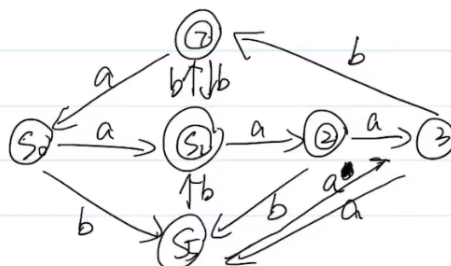
$$\{S_5\}a = \{S_5\}$$

$$\{S_1\}a = \{S_2\}$$

$$\{S_2\}a = \{S_4\}$$

$$\{S_7\}a = \{S_0\}$$

$$\pi_{\text{new}} = \{ \{S_0\} \{S_1\} \{S_5\} \{S_2\} \{S_4\} \{S_7\} \}$$



首先将其中根本就到不了的结点直接删除。

正规语言的DFA

3-22 分别构造识别如下正规语言的 DFA:

(1) $((0^* | 1)(1^* 0))^*$

△ 正规式: 将文法的终结符号, 用 $*$, $.$, $|$ 连接组成的表达式

△ 由正规式构造 DFA (分裂法)

① $i \xrightarrow{r.s} j$

$i \xrightarrow{r} k \xrightarrow{s} j$

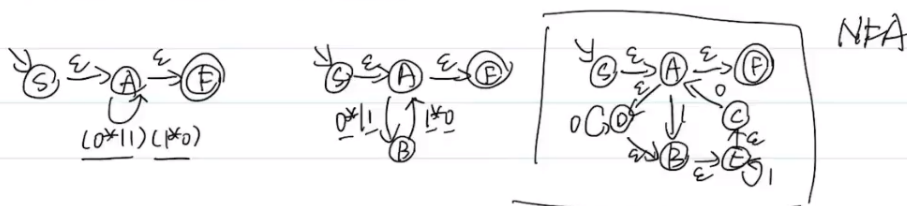
② $i \xrightarrow{r|s} j$

$i \xrightarrow{r} j$
 $i \xrightarrow{s} j$

③ $i \xrightarrow{r^*} j$

$i \xrightarrow{\epsilon} k \xrightarrow{\epsilon} j$
 $k \xrightarrow{r} k$

构造后进行确定化和最小化



确定化 ① $\epsilon\text{-CLOSURE}(S) = \{S, A, B, C, D, E, F\}$

② I	I ₀	I ₁
→ $\{S, A, B, C, D, E, F\}$	$\{A, F, D, B, E, C\}$ q_1	$\{B, E, C\}$ q_2
← $\{A, F, D, B, E, C\}$	q_1	q_2
← $\{B, E, C\}$	q_1	$\{E, C\}$ q_3
← $\{E, C\}$	q_1	q_3

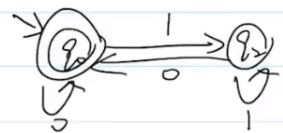
最小化: $\pi = \{\{q_0, q_1\}, \{q_2, q_3\}\}$

$\{q_0, q_1\}_0 = \{q_1\}$, 不可区分 $\Rightarrow$ 合并 q_0

$\{q_0, q_1\}_1 = \{q_2\}$

$\{q_2, q_3\}_0 = \{q_1\}$, 不可区分 $\Rightarrow$ 合并 q_2

$\{q_2, q_3\}_1 = \{q_3\}$



语法分析和语法分析程序

△ 自顶向下的语法分析 { LL(1)分析 → LL(1) ✓
 递归下降分析
 预测分析

自底向上的语法分析 { 简单优先
 算符优先 ✓
 优先函数
 LR分析: LR(0) < SLR(1) < LALR(1) < LR(1) < 无歧义性

消除左递归式

4-1 消除下列文法的左递归性。

(3) $S \rightarrow (T)$ $S \rightarrow a$ $S \rightarrow \epsilon$
 $T \rightarrow S$ $T \rightarrow T, S$

△ 消除左递归性:

$A \rightarrow A\alpha | \beta \Rightarrow \begin{cases} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' | \epsilon \end{cases}$

$T \rightarrow T(S) | (S) \Rightarrow T \rightarrow ST'$
 $T' \rightarrow ,ST' | \epsilon$

$S \rightarrow (T)$ $S \rightarrow a$ $S \rightarrow \epsilon$ $T \rightarrow ST'$ $T' \rightarrow ,ST' | \epsilon$

其中a是与自身连接的符号，b为非自身的符号，不管是否存在非终结符。

First和Follow集

△ FIRST 集

1) x 是终结符, $FIRST(x) = \{x\}$

2) x 是非终结符, $\begin{cases} x \rightarrow a\alpha, \text{ 则 } a \in FIRST(x) \\ x \rightarrow \epsilon, \text{ 则 } \epsilon \in FIRST(x) \end{cases}$

3) $x \rightarrow Y_1 Y_2 \dots Y_k$

① Y_1 是非终结符, $FIRST(Y_1) - \{\epsilon\} \in FIRST(x)$

② $Y_1, Y_2, \dots, Y_i (1 \leq i \leq k-1)$ 均为非终结符且都能推出 ϵ ,

则 $FIRST(Y_1), FIRST(Y_2), \dots, FIRST(Y_{i+1})$ 中一切非 $\epsilon \in FIRST(x)$

③ $Y_1, Y_2, \dots, Y_k$ 都能推导出 ϵ , 则 $\epsilon \in FIRST(x)$

△ FOLLOW 集 (A 为非终结符)

1) 开始符 S , $\# \in FOLLOW(S)$

2) $B \rightarrow \alpha A \beta$, $FIRST(\beta) - \{\epsilon\} \in FOLLOW(A)$

3) $B \rightarrow \alpha A \beta$ 且 $\epsilon \in FIRST(\beta)$, 则 $FOLLOW(B) \in FOLLOW(A)$
 $\quad \quad \quad B \rightarrow \alpha A$

4-4 对于如下的文法, 求出各个 FIRST 集和 FOLLOW 集。

$S \rightarrow aAB$

$S \rightarrow bA$

$S \rightarrow \epsilon$

$A \rightarrow aAb$

$A \rightarrow \epsilon$

$B \rightarrow bB$

$B \rightarrow \epsilon$

产生式	FIRST 集	FOLLOW 集
<u>$S \rightarrow aAB$</u>	a	$FOLLOW(S) = \#$
<u>$S \rightarrow bA$</u>	b	
<u>$S \rightarrow \epsilon$</u>	ϵ	
<u>$A \rightarrow aAb$</u>	a	$FOLLOW(A) = b, \#$
<u>$A \rightarrow \epsilon$</u>	ϵ	
<u>$B \rightarrow bB$</u>	b	$FOLLOW(B) = \#$
<u>$B \rightarrow \epsilon$</u>	ϵ	

First集就是推导式最左边的截止符。Follow集，每个状态follow集是同一个，找到存在这个状态的产生式，然后找它右边的元素，其First集中非空元素即为Follow集中的元素；如果他后面没有元素，则自身的Follow集中加入右边元素的Follow集。

LL(1) 文法

判断LL(1)文法

4-7 验证下列文法是否为 LL(1) 文法。

(1) $S \rightarrow AB$ $S \rightarrow CDa$ $A \rightarrow ab$
 $A \rightarrow c$ $B \rightarrow dE$ $C \rightarrow eC$
 $C \rightarrow \epsilon$ $D \rightarrow fD$ $D \rightarrow f$
 $E \rightarrow dE$ $E \rightarrow \epsilon$

不是LL(1)文法 $\because FIRST(fD) \cap FIRST(f) = f \neq \emptyset$

(2) $S \rightarrow aABbCD$ $S \rightarrow \epsilon$ $A \rightarrow ASd$
 $A \rightarrow \epsilon$ $B \rightarrow SAc$ $B \rightarrow eC$
 $B \rightarrow \epsilon$ $C \rightarrow Sf$ $C \rightarrow Cg$
 $C \rightarrow \epsilon$ $D \rightarrow aBD$ $D \rightarrow \epsilon$

不是LL(1)文法 $\because$ 存在左递归

△判断LL(1)文法:
 1) 已化简且无左递归
 2) 对G中每个产生式 $A \rightarrow r_1 | r_2 | \dots | r_m$, 其各候选式均满足:
 ① $FIRST(r_i) \cap FIRST(r_j) = \emptyset$ (两两不相交)
 ② 若有候选式为 ϵ , 则 $FIRST(r_i) \cap FOLLOW(A) = \emptyset$
 $A \rightarrow \epsilon$ 其余候选式

LL(1)分析表

4-8 对于如下的文法 $G[S]$:

$S \rightarrow Sb$ $S \rightarrow Ab$ $S \rightarrow b$
 $A \rightarrow Aa$ $A \rightarrow a$

(1) 构造一个与 G 等价的 LL(1) 文法 G' ;
 (2) 对于 G' , 构造相应的 LL(1) 分析表。

(1) $S \rightarrow Sb | Ab | b \Rightarrow S \rightarrow \underline{A}bS' | \underline{b}S'$
 $A \rightarrow Aa | a \Rightarrow A \rightarrow \underline{a}A'$
 $A' \rightarrow \underline{a}A' | \epsilon$

△LL(1)分析表

非终结符 | 终结符 + (#)

对 $A \rightarrow r_i$
 ① 求 $FIRST(r_i) = a$, 则 $M[A, a]$ 处填入此产生式.
 ② 若 $FIRST(r_i) = \epsilon$, 则求 $FOLLOW(A) = \underline{b}$, $A \rightarrow \epsilon$
 则 $M[A, b]$ 处填入此产生式
 ③ 其余空着. error

(2)

	a	b	#
S	$S \rightarrow \underline{A}bS'$	$S \rightarrow \underline{b}S'$	
S'		$S' \rightarrow \underline{b}S'$	$S' \rightarrow \epsilon$
A	$A \rightarrow \underline{a}A'$		
A'	$A' \rightarrow \underline{a}A'$		

$FIRST(S \rightarrow \underline{A}bS') = a$ $FIRST(\underline{b}S') = b$
 $FIRST(S' \rightarrow \underline{b}S') = b$ $\#$
 $\because S' \rightarrow \epsilon$ $FOLLOW(S') = \#$
 $FIRST(A \rightarrow \underline{a}A') = a$ $FIRST(A' \rightarrow \underline{a}A') = a$
 $\therefore A' \rightarrow \epsilon$ $FOLLOW(A')$

LR(0)

1. 得到拓广文法，将给的文法前面加上 $S' \rightarrow S$ ，并对文法依次进行编号。
2. 识别全部活前缀的DFA。其实就是一个挪动 $\cdot$ 向后的过程，直到 $\cdot$ 到达表达式就成为归约项目。方法: 1. 标注状态I 2. 写前面留存下来的表达式 3. 表达式如果右端有非终结符，则将非终结符写在下面 4. $\cdot$ 向后移跨过字符就是状态转换的条件 5. 直到将所有表达式转成归约项目 6. 如果转化状态之前就有就往回画。
3. 项目集规范族 $\{I_0, I_1, \dots, I_n\}$ ，项目集是指所有等价项目的集合。
4. 没有归约冲突和移进冲突，则为LR(0)文法
5. LR(0)分析表 1. Action中写终结符号和# Goto中写非终结符 2. Action是填其他状态，Goto是填到达归约项目的状态，填数字 3. S' 为接受项目，在Action中的#写入acc 4. 归约项目填表，是写rx，x为是由拓广文法哪个推导式出来的序号。

4-35 对于下列的文法，试分别构造它们的 LR(0)项目集规范族及识别全部活前缀的 DFA。

4-36 对于习题 4-35 中的各文法，判别哪些是 LR(0) 文法，并对它们构造相应的 LR(0) 分析表。

(2) $S \rightarrow cA$ $S \rightarrow ccB$ $A \rightarrow cA$
 $A \rightarrow a$ $B \rightarrow ccB$ $B \rightarrow b$

Δ LR(0) 项目: ① 归约项目 ($A \rightarrow \alpha \cdot$) ② 接受项目 ($S \rightarrow \alpha \cdot$)

↓ ③ 移进项目 ($A \rightarrow \alpha \cdot X\beta$) ④ 待约项目 ($A \rightarrow \alpha \cdot X\beta$)

指在部某位置上标有 $\cdot$ 的产生式 eg. $A \rightarrow xy\cdot$ 有约项目 $A \rightarrow \cdot xy\cdot$ $A \rightarrow x\cdot y\cdot$ $A \rightarrow xy\cdot\cdot$ $A \rightarrow xy\cdot\cdot$

Δ 所有等价项目组成一个项目集 I ，称为项目集闭包，每个项目集闭包对应自动机的一个状态

Δ 项目集的构造: 求基本项目集的闭包 $\Rightarrow$ 找等价项目

eg. $S \rightarrow cc\cdot B$ $B \rightarrow \cdot ccB$ $B \rightarrow \cdot b$

Δ 判断 LR(0) 文法: 无冲突项目 (移进-归约, 归约-归约冲突)

Δ LR(0) 分析表:

	ACTION 终结符 + #	GOTO 非终结符
状态 n		

ACTION: SN : 遇此终结符移到 n 状态

rN : 此状态 n 为归约项目, 此行都填 rN

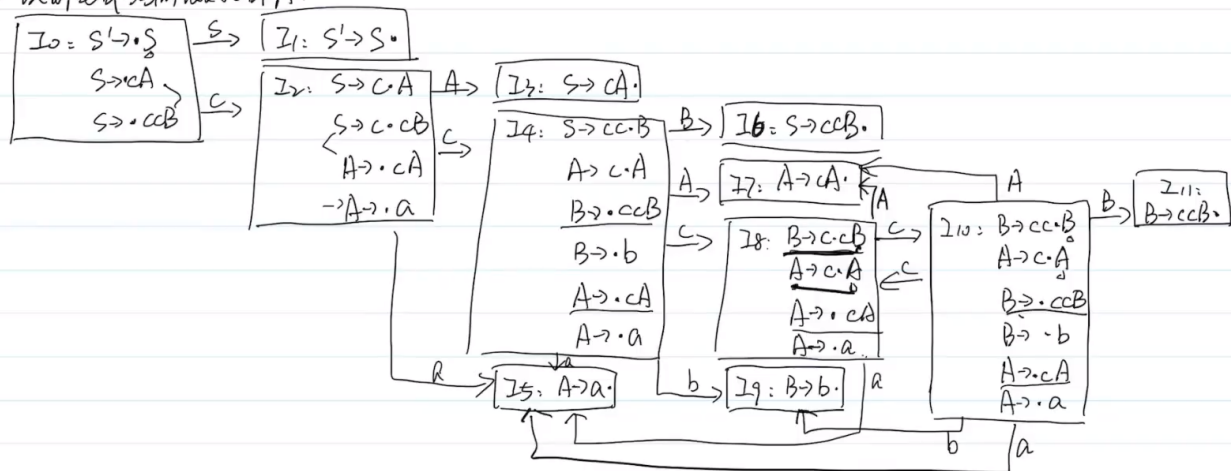
acc : 接受项目, #处填 acc $S \rightarrow cA\cdot$

$GOTO$: 填 n , 经过此非终结符到达的状态 n

① 拓广文法: ② $S' \rightarrow S$ ③ $S \rightarrow cA$ ④ $S \rightarrow ccB$ ⑤ $A \rightarrow cA$ ⑥ $A \rightarrow a$ ⑦ $B \rightarrow ccB$ ⑧ $B \rightarrow b$

① 拓广文法: ① $S' \rightarrow S$ ② $S \rightarrow cA$ ③ $S \rightarrow cCB$ ④ $A \rightarrow cA$ ⑤ $A \rightarrow a$ ⑥ $B \rightarrow cCB$ ⑦ $B \rightarrow b$

② 识别全部活前缀的DFA.



项目集规范族 $\{I_0, I_1, I_2, \dots, I_{11}\}$

是 LR(0) 文法

LR(0) 分析表

	ACTION					GOTO		
	a	b	c	#		S	A	B
0			S ₂			1		
1				acc				
2	S ₅		S ₄				3	
3	r ₁	r ₁	r ₁	r ₁				
4	S ₅	S ₉	S ₈				7	6
5	r ₄	r ₄	r ₄	r ₄				
6	r ₂	r ₂	r ₂	r ₂				
7	r ₃	r ₃	r ₃	r ₃				
8	S ₅		S ₁₀				7	
9	r ₆	r ₆	r ₆	r ₆				
10	S ₅	S ₉	S ₈				7	11
11	r ₅	r ₅	r ₅	r ₅				

SLR(1) 文法

4-38 下列文法是否为 SLR(1) 文法? 若是, 构造相应的 SLR(1) 分析表, 若不是, 则阐明其理由。

△ 判断 SLR(1) 文法

① DFA 中存在冲突项目 (移进-归约, 归约-归约)

② $I_n = \{ A_1 \rightarrow \alpha \cdot a_1 \beta, A_2 \rightarrow \alpha \cdot a_2 \beta \dots, \underbrace{B_1 \rightarrow \alpha \cdot}_{\text{移进项目}}, \underbrace{B_2 \rightarrow \alpha \cdot \dots}_{\text{归约项目}} \}$

当 $\{a_1, a_2 \dots a_m\}, FOLLOW(B_1), FOLLOW(B_2) \dots$ 两两不相交时。

为 SLR(1) 文法

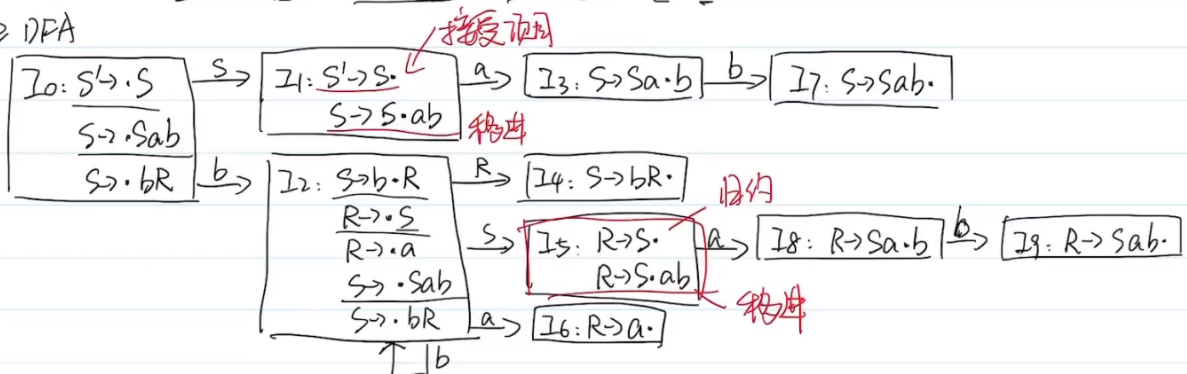
△ SLR(1) 分析表

对每个归约项目 $A \rightarrow \alpha \cdot$, 求 $FOLLOW(A)$ 得到对输入符号处填 r_n
其余同 LR(0) 分析表一样。

(1) $S \rightarrow Sab$ $S \rightarrow bR$ $R \rightarrow S$ $R \rightarrow a$

① 拓广文法: ① $S' \rightarrow S$ ② $S \rightarrow Sab$ ③ $S \rightarrow bR$ ④ $R \rightarrow S$ ⑤ $R \rightarrow a$

② DFA



不存在移进-归约冲突

$\{a\} \cap FOLLOW(R) = FOLLOW(S) = \{a, \# \} \neq \emptyset$

不是 SLR(1) 文法

LR(1) 文法

$S \rightarrow A$ $B \rightarrow aB$ $A \rightarrow BA$ $B \rightarrow b$ $A \rightarrow \epsilon$

(1) 构造 LR(1) 分析表;

(2) 给出用 LR(1) 分析表对输入字符串 abab 的分析过程。

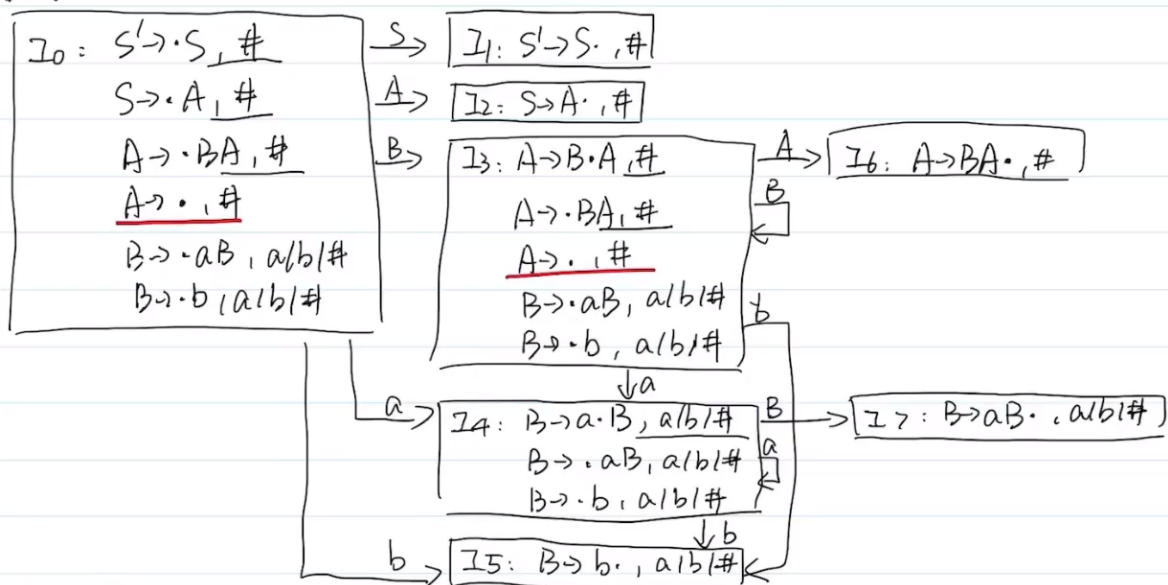
① 该文法:

② $S' \rightarrow S$ ③ $S \rightarrow A$ ④ $A \rightarrow BA$ ⑤ $A \rightarrow \epsilon$ ⑥ $B \rightarrow aB$ ⑦ $B \rightarrow b$ Δ 判断 LR(1) 文法

构造带向前搜索的 DFA, 无归约-归约冲突。

 Δ LR(1) 文法 DFA在 I_n 中, 每一项目开始如 $A \rightarrow \alpha \cdot \beta, a$ $A \rightarrow \alpha \cdot B \beta, a$ 其等价项目 $B \rightarrow \cdot r, b$ 中 $b = \text{FIRST}(\beta a)$ $\left\{ \begin{array}{l} \beta = \epsilon \text{ 时, } b = a \text{ 继承} \\ \beta \neq \epsilon \text{ 时, } b = \text{FIRST}(\beta) \end{array} \right.$ Δ LR(1) 分析表每个状态 I_n 中, 若存在归约项目。其 m 根据向前搜索项填写

DFA:



LR(1) 分析表	ACTION			GOTO		
	a	b	#	S	A	B
0	<u>S4</u>	S5	r3	1	2	3
1			acc			
2			r1			
3	S4	S5	r3		6	3
4	S4	S5				7
5	r5	r5	r5			
6			r2			
7	r4	r4	r4			

分析过程		abab		ACTION	GOTO
步骤	状态	栈中符号	余留符号串	分析动作	T-状态
1	0	#	<u>a</u> bab#	S4	4
2	04	#a	<u>b</u> ab#	S5	5
3	045	#ab	<u>a</u> b#	r5	GOTO[4, B]=7
	047	#aB	<u>a</u> b#	r4	GOTO[0, B]=3
	03	#B	<u>a</u> b#	S4	4
	034	#Ba	<u>b</u> #	S5	5
	0345	#Bab	#	r5	GOTO[4, B]=7
	0347	#BaB	#	r4	GOTO[3, B]=3
	033	#BB	#	r3	GOTO[3, A]=6
	0336	#BBA	#	r2	GOTO[3, A]=6
	036	#BA	#	r2	GOTO[0, A]=2
	02	#A	#	r1	GOTO[0, S]=1
	01	#B	#	acc	

代码优化

DAG

◇ 基本块的 DAG 表示

– DAG

DAG 指有向无圈图 (*Directed Acyclic Graph*)

– 基本块的 DAG 是在结点上带有标记的 DAG

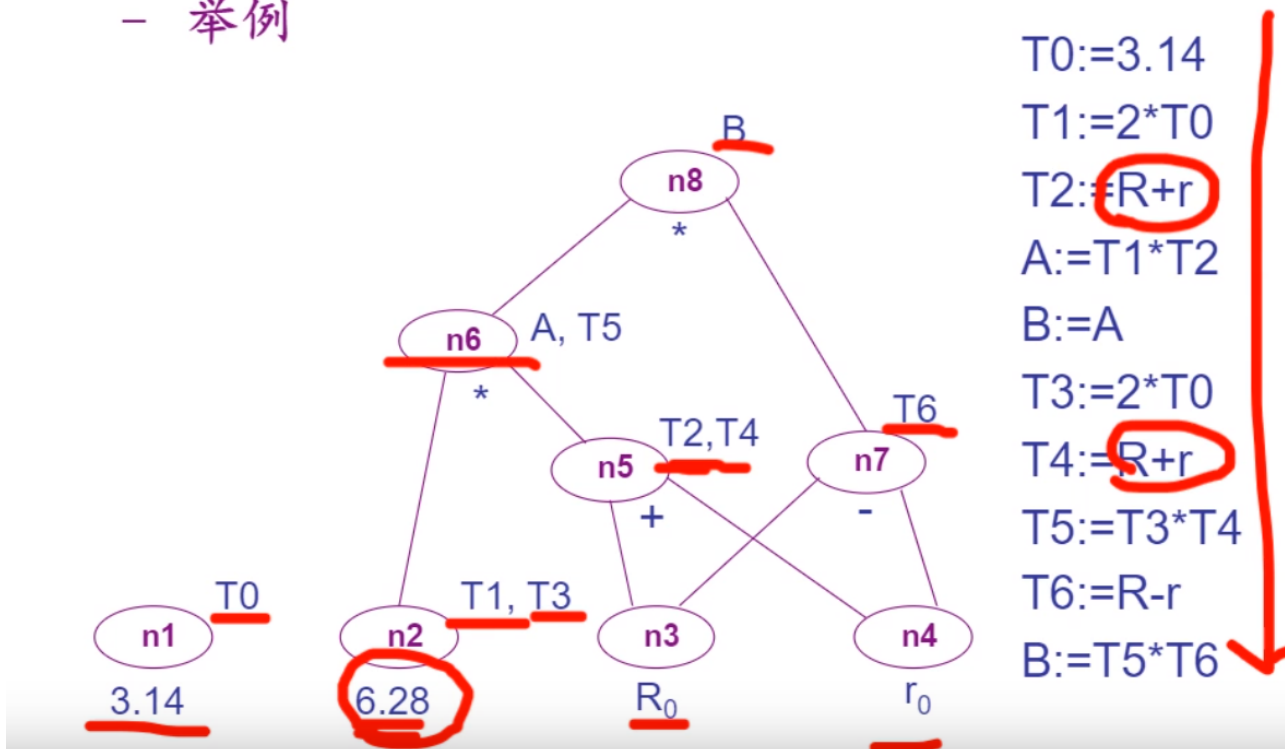
叶结点 代表名字的初值，以唯一的标识符（变量名字或常数）标记（为避免混乱，用 x_0 表示变量名字 x 的初值）

内部结点 用运算符号标记

所有结点都可有一个附加标识符表

◇ 基本块 DAG 表示的构造

- 举例



◇ 从基本块的 DAG 表示可得到等价的基本块

- 比较变换前后的基本块

